SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4303
MARK: 92/100

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: **Total Marks: 30**

Code of Conduct:

I SOFIA RV - 192511055(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: SOFIA RV

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. **Identify appropriate HTML5 semantic elements missing in the structure.**
2. **Modify the structure to make it standards-compliant.**
3. **Interpret the HTTP request generated during form submission.**
4. **Analyze the HTTP response cycle from server to browser.**
5. **Suggest improvements for usability and accessibility.**

ANSWER

1. Identify appropriate HTML5 semantic elements missing in the structure.

The given HTML form lacks appropriate semantic elements that improve the structure and accessibility of the webpage. The following elements can be included:

- <header> – Displays the symposium title and introductory information.
- <nav> – Provides navigation links to different pages.
- <main> – Contains the main registration content.
- <section> – Groups related registration information.
- <footer> – Contains copyright and contact information.
- <label> – Provides descriptive names for input fields.
- <fieldset> and <legend> – Group related form controls and provide a heading for the group.

These elements improve the readability, accessibility, and maintainability of the webpage.

2. Modify the structure to make it standards-compliant.

The following HTML5 code provides a valid document structure with semantic elements and accessible form controls.

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Online Symposium Registration</title>
</head>
<body>

<header>
    <h1>Online Symposium Registration</h1>
</header>

<main>
    <section>
        <h2>Student Registration</h2>

        <form action="/register" method="POST">
            <fieldset>
                <legend>Registration Details</legend>

                <label for="name">Student Name:</label>
                <input type="text" id="name" name="name" required>

                <br><br>
                <label for="email">Email Address:</label>
                <input type="email" id="email" name="email" required>
                <br><br>

                <button type="submit">Register</button>
            </fieldset>
        </form>
    </section>
</main>
<footer>
    <p>College Online Symposium</p>
</footer>
</body>
</html>
```

The code uses the HTML5 doctype, semantic elements, proper labels, appropriate input types, and required-field validation. These features improve standards compliance and accessibility.

3. Interpret the HTTP request generated during form submission.

When the student enters the name and email address and clicks the Submit button, the browser generates an HTTP POST request and sends it to the /register endpoint specified in the form.

Example HTTP request:

```
POST /register HTTP/1.1
Host: www.college.com
Content-Type: application/x-www-form-urlencoded
name=Rahul&email=rahul%40example.com
```

The components are explained below:

- POST: Specifies the HTTP method used to submit data to the server.
- /register: Identifies the server resource that receives the registration data.
- Host: Specifies the destination web server.
- Content-Type: Indicates the format of the submitted form data.
- Request body: Contains the student's name and email address.

The server receives the request and processes the submitted information. The POST method is suitable for submitting registration data because the form values are transmitted in the request body rather than appended to the URL.

4. Analyze the HTTP response cycle from server to browser.

The HTTP response cycle involves communication between the browser and the web server through the following steps:

1. Form submission: The student enters the required information and clicks Submit.
2. HTTP request: The browser sends a POST request containing the form data to the /register endpoint.
3. Server processing: The server receives the request, validates the data, and processes the registration.
4. HTTP response: The server returns a response containing a status code, headers, and optionally a response body.
5. Browser processing: The browser receives the response and interprets the status code and content.
6. Result display: The browser displays a success message or an appropriate error message to the student.

Example successful response:

```
HTTP/1.1 200 OK
Content-Type: text/html
```

The response body may contain an HTML page displaying “Registration Successful.”

The HTTP status code 200 OK indicates that the request was successful.

HTTP communication cycle:

Student submits the form

↓

Browser sends POST request

↓

Web server processes data

↓

Server sends HTTP response

↓

Browser receives the response

↓

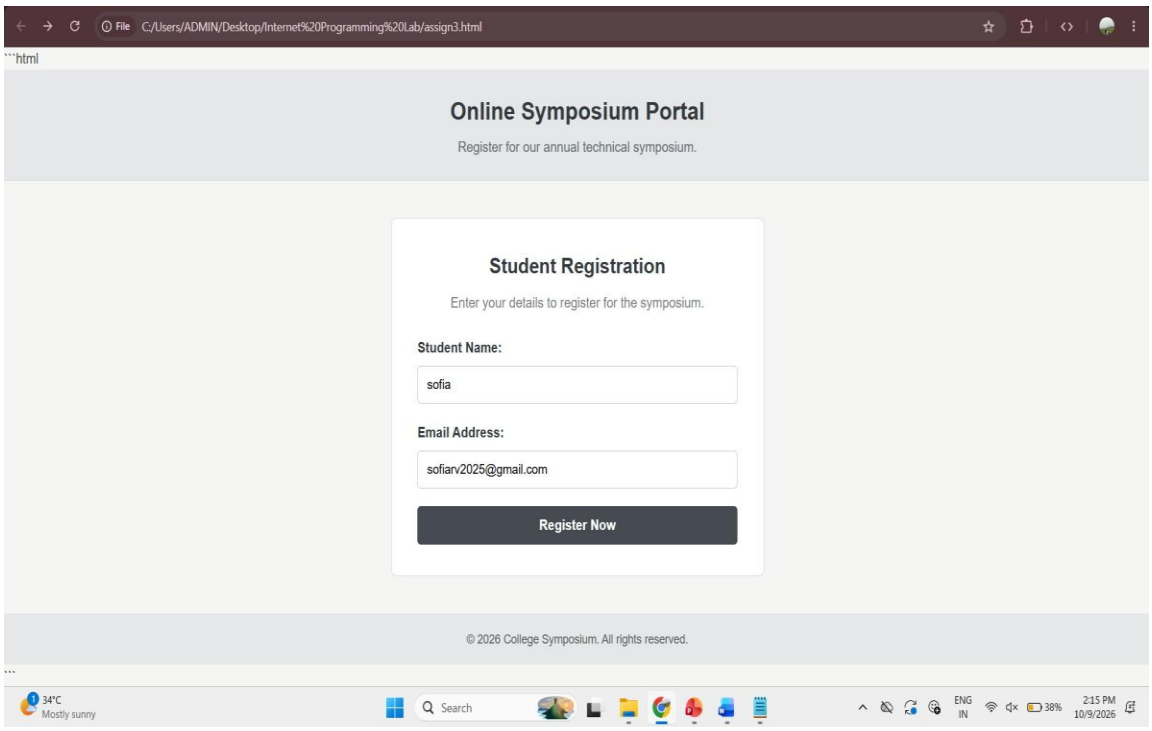
Result displayed to student

5. Suggest improvements for usability and accessibility.

The following improvements can make the symposium registration portal more user-friendly and accessible:

1. Proper labels: Associate every input field with a <label> element to help users and screen readers identify the required information.
2. Form validation: Use the required attribute and appropriate input types, such as type="email", to prevent incomplete or incorrectly formatted submissions.
3. Clear instructions: Provide instructions explaining what information students must enter.
4. Meaningful error messages: Display specific messages when a field is empty or an email address is invalid.
5. Descriptive buttons: Use clear button text such as “Register for Symposium” instead of a generic “Submit.”
6. Keyboard accessibility: Ensure that all form fields and buttons can be accessed and operated using the keyboard.
7. Sufficient color contrast: Use readable text and background colors to support users with visual difficulties.
8. Responsive design: Ensure that the form adapts to desktop, tablet, and mobile screens.
9. Data security: Use HTTPS to encrypt information transmitted between the browser and server.
10. Accessible layout: Use semantic HTML elements and a logical heading structure to improve navigation and compatibility with assistive technologies.

These improvements enhance usability, accessibility, security, and the overall registration experience.



2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Questions:

1. **Identify improper nesting and missing closing tags.**
2. **Analyze how different browsers may interpret this structure.**
3. **Identify missing semantic elements (e.g., header, section).**
4. **Provide a corrected W3C-compliant HTML structure.**
5. **Suggest techniques to ensure cross-browser compatibility.**

ANSWER

1. Identify improper nesting and missing closing tags.

The given HTML code contains several structural errors:

- The <!DOCTYPE html> declaration is missing.
- The <html> element does not specify the document language.
- The <meta charset="UTF-8"> declaration is missing.
- The <h1> element is missing its closing tag </h1>.
- The element is missing its closing tag .
- The contains improperly terminated elements, resulting in an incomplete HTML structure.

These errors can cause inconsistent rendering and make the webpage difficult to maintain.

2. Analyze how different browsers may interpret this structure.

Different browsers may handle malformed HTML differently because they use HTML parsing rules to recover from errors.

- Automatic tag correction: Browsers may automatically insert or infer missing closing tags.
- Different rendering behavior: The layout and spacing of headings and paragraphs may vary depending on how the browser repairs the document.
- DOM differences: The browser-generated Document Object Model (DOM) may differ from the intended structure.
- Unexpected layout: Missing tags may cause content to be displayed incorrectly or affect CSS styling.
- Compatibility issues: The page may behave inconsistently across browsers such as Chrome, Firefox, Safari, and Edge.

Therefore, properly structured HTML is essential for consistent rendering across browsers.

3. Identify missing semantic elements (e.g., header, section).

The given code uses only generic elements and lacks appropriate semantic elements.

- <header> – Contains the company name, logo, or introductory content.
- <main> – Contains the primary content of the webpage.
- <section> – Groups related content, such as the company introduction.
- <footer> – Contains copyright information and contact details.
- <nav> – Provides navigation links when required.

These elements improve accessibility, readability, document organization, and search engine understanding.

4. Provide a corrected W3C-compliant HTML structure.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Company</title>
</head>
<body>

   <header>
      <h1>Welcome</h1>
   </header>

   <main>
      <section>
         <h2>About Us</h2>
         <p>Learn more about our company and services.</p>
      </section>
   </main>

   <footer>
      <p>&copy; 2026 Company. All rights reserved.</p>
   </footer>

</body>
</html>
```

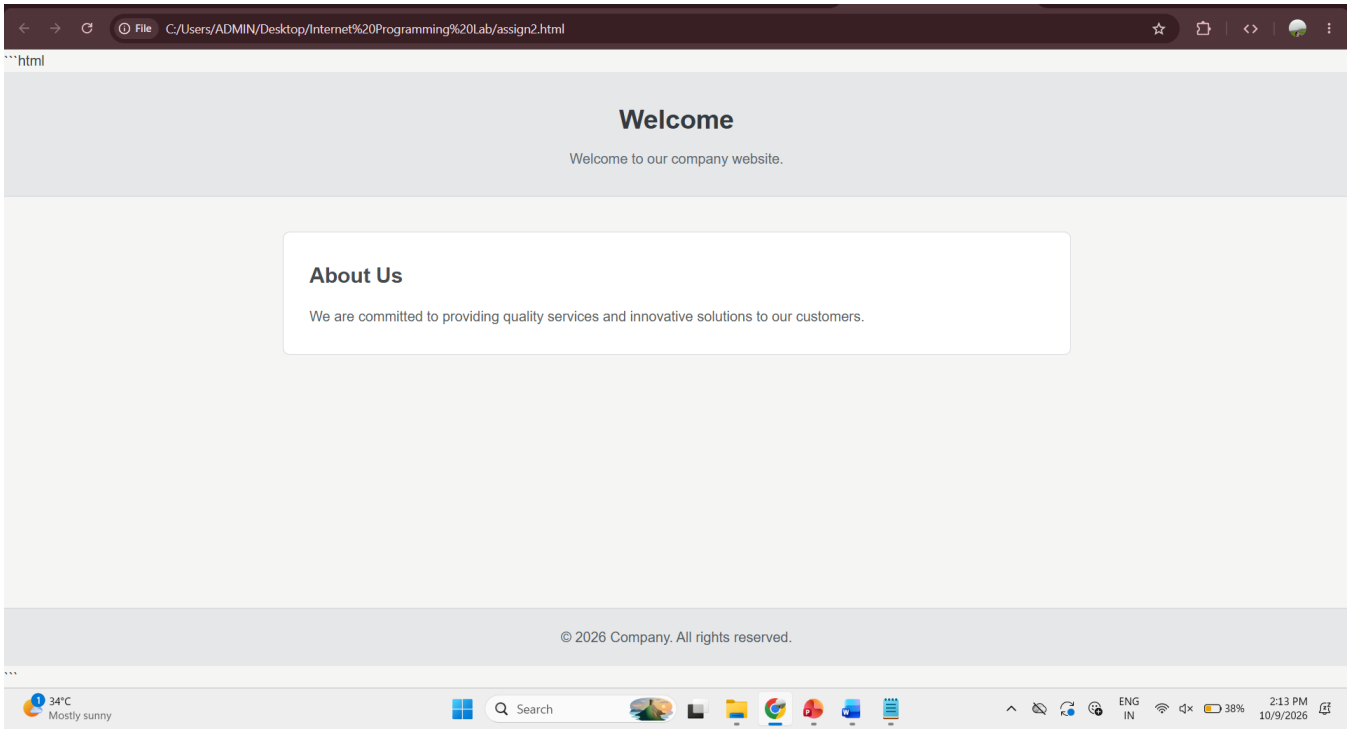
The corrected code includes the HTML5 doctype, character encoding, viewport settings, semantic elements, and properly nested and closed tags. It follows standard HTML structure and can be validated using the W3C Markup Validation Service.

5. Suggest techniques to ensure cross-browser compatibility.

The following techniques help maintain consistent webpage behavior across browsers:

1. Use valid HTML: Follow HTML5 standards and close elements correctly.
2. Validate the code: Use the W3C Markup Validation Service to identify HTML errors.
3. Use CSS resets or normalization: Reduce differences in default browser styles.
4. Follow responsive design: Use flexible layouts, media queries, and relative units to support different screen sizes.
5. Test across browsers: Check the webpage in Chrome, Firefox, Safari, and Edge.
6. Use standard CSS properties: Avoid unnecessary browser-specific properties and outdated features.
7. Specify character encoding: Use <meta charset="UTF-8"> to ensure consistent text display.
8. Use feature detection: Check whether a browser supports a feature before depending on it.
9. Maintain semantic HTML: Use appropriate elements to ensure consistent structure and accessibility.
10. Test on different devices: Verify the layout on desktops, tablets, and mobile devices.

These techniques improve browser compatibility, accessibility, maintainability, and the overall user experience.



3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. **Identify issues in the form configuration.**
2. **Analyze why the HTTP request is not triggered.**
3. **Interpret the role of GET method in this scenario.**
4. **Propose a corrected implementation.**
5. **Suggest improvements for secure data transmission.**

ANSWERS

1. Identify issues in the form configuration.

The given form contains the following issues:

- The form uses the GET method, which is unsuitable for transmitting sensitive information such as passwords.
- The button has type="button" instead of type="submit", so clicking it does not automatically submit the form.
- The form does not include the required attribute to ensure that the username and password are entered.
- No client-side validation or user feedback is provided.

These issues can prevent form submission and create security risks.

2. Analyze why the HTTP request is not triggered.

The main reason is that the button is defined as:

<button type="button">Login</button>

The type="button" attribute creates a normal button that does not submit the form automatically.

When the user clicks the button, the browser does not send an HTTP request unless JavaScript is used to handle the click and initiate submission.

Changing the button type to submit allows the browser to submit the form using the specified action and method.

3. Interpret the role of GET method in this scenario.

The GET method retrieves information from a server. When used in a form, the browser appends the submitted form data to the URL as query parameters.
For example:
/submit?username=rahul&password=12345
The GET method is unsuitable for login forms because:

- Passwords may appear in browser history and URLs.
- URLs may be stored in server logs and other records.
- URLs may be exposed through shared links or screenshots.
- The request URL is not intended for transmitting sensitive credentials.

Therefore, the POST method should be used for submitting login credentials. However, POST alone does not encrypt data; HTTPS is also required.

4. Propose a corrected implementation.

The following HTML code corrects the form submission problem:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Login Form</title>
</head>
<body>

   <h1>User Login</h1>

   <form action="/submit" method="POST">

      <label for="username">Username:</label>
      <input type="text" id="username"
            name="username" required>

      <br><br>

      <label for="password">Password:</label>
      <input type="password" id="password"
            name="password" required>
```

```
        <br><br>

        <button type="submit">Login</button>

    </form>

</body>
</html>
```

Explanation:

- method="POST" sends the form data in the request body.
- type="submit" triggers form submission.
- required ensures that both fields are filled in.
- <label> elements improve accessibility.
- type="password" masks the password characters on the screen.

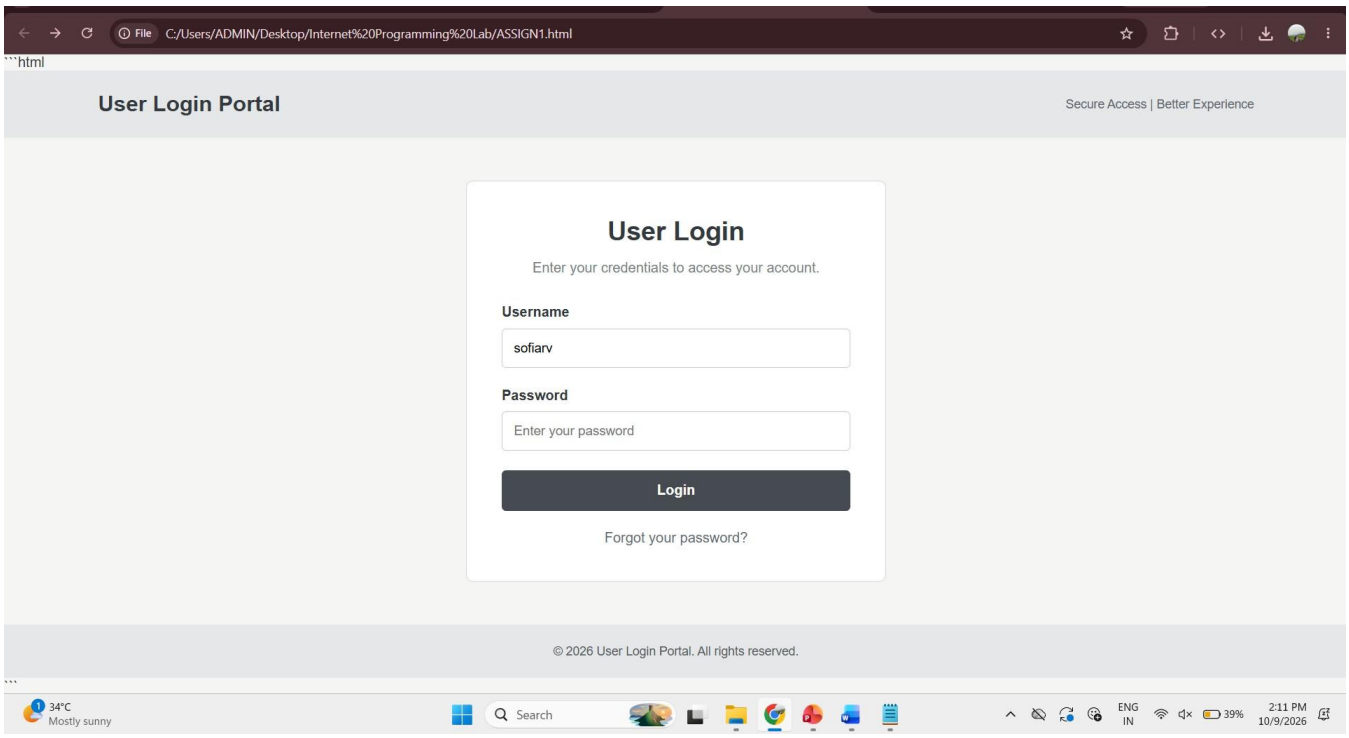
The server must validate the submitted credentials and return an appropriate response.

5. Suggest improvements for secure data transmission.

The following measures improve login security:

1. Use HTTPS: Encrypts data transmitted between the browser and server.
2. Use the POST method: Prevents credentials from being placed directly in the URL.
3. Validate input: Check the submitted username and password on both the client and server.
4. Hash passwords securely: Store passwords using a suitable password-hashing algorithm such as Argon2id or bcrypt, rather than storing plaintext passwords.
5. Use secure session management: Generate unpredictable session identifiers and use secure session cookies.
6. Prevent brute-force attacks: Apply rate limiting and temporary delays after repeated failed login attempts.
7. Avoid exposing credentials: Never include passwords in URLs, error messages, or application logs.
8. Implement server-side authentication: Verify credentials on the server before granting access.

These measures help prevent credential exposure, unauthorized access, and other security vulnerabilities.


File C:/Users/ADMIN/Desktop/Internet%20Programming%20Lab/ASSIGN1.html
```html
User Login Portal
Secure Access | Better Experience
User Login
Enter your credentials to access your account.
Username
sofiarv
Password
Enter your password
Login
Forgot your password?
© 2026 User Login Portal. All rights reserved.
```
34°C
Mostly sunny
Search
ENG
IN
39%
2:11 PM
10/9/2026

RUBRICS FOR CALCULATION:

Criteria	Weig htage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconception s	Poor understandin g; major misconceptio ns
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

AWARDED MARK

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**	**Mark Awarded**
Understanding of Concepts	25%	Demonstrates thorough and accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding with major misconceptions	23/25
Explanation	25%	Provides detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation lacking depth	Poor or unclear explanation	23/25
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis	18/20
Organization	15%	Presents well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure lacking clarity	Poor organization and incoherent answers	14/15
Accuracy	10%	Provides accurate answers covering all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps	9/10
Clarity	5%	Maintains clear, neat, and professional presentation	Understandable with minor issues	Limited clarity and readability issues	Poorly written and difficult to understand	5/5
Total	**100%**				**Overall Mark**	**92/100**